CHAPTER 1

INTRODUCTION

Each day brings more technological progress. Today's company market is competitive, and everyone strives to give customers the best options. Some people sell unlicensed things to get rich quickly. The Organisation for Economic Co-operation and Development (OECD) and the European Union Intellectual Property Office (2019) define counterfeit goods as products that copy the appearance of genuine goods without permission . Recent years have seen a boom in counterfeit goods, and the spectrum of products in danger has grown. With almost half of US Customs and Border Protection's counterfeit items being clothes and accessories, copyright violations have increased. While international trade has remained flat, counterfeit goods have grown to 3.3% . Over the past decade, counterfeit products have flooded Bangladesh's market. The National Consumers' Rights Protection Directorate identified counterfeit powdered milk and cosmetics in large stores in Banani, Gulshan, and other affluent areas during enforcement operations. During the raid, officers found that several facilities were making counterfeit consumer products without MRP, importer names, or addresses. Even during the COVID-19 pandemic, dishonest businessmen created fake masks, gloves, and sanitisers. A group of dishonest merchants has started selling phoney and low-quality goods during the coronavirus crisis. Famous national and international brands are being forged alarmingly. This includes food, medicine, and life-saving items.

Blockchain technology is a popular topic among intellectuals. Blockchain has emerged in the supply chain, pharmaceutical, and health sectors. No third-party participation is a benefit of decentralized blockchain design. Blockchain has several advantages and can be used in industries where manufacturers can create blockchain-based QR codes, and customers can obtain real products. Manufacturers can transfer product ownership to sellers using blockchain technology, and buyers can verify product legitimacy before buying. Blockchain technology secures digital asset ownership certificates and preserves their physical properties. Blockchain makes it difficult for an attacker to edit, change, or replicate this data. Considering these prospects, a system architecture is developed that uses blockchain technology to protect product ownership and lets customers pick between counterfeit and authentic Ethereum smart contract products. This technology lets users scan

products using blockchain-based QR codes to verify authenticity, and it reduces manufacturing costs.This project aims to provide a solution to the growing problem of product counterfeiting and enhance the transparency and trustworthiness of supply chains using blockchain technology.

1.1 Organization of the Project Report

Chapter 1: Introduction

It includes a brief introduction about the existing system and proposed system also the advantages and disadvantages.

Chapter 2: Literature Survey

This include a brief information about the survey papers collected to gain the information for the project, like what are all the methods and protocols used previously.

Chapter 3: System Requirements and Specifications

This includes the required specification for project like functional requirements, non-functional requirements, hardware requirements and software requirements.

Chapter 4: System Design

This includes the complete description of the project and also explain about the architecture of the system.

Chapter 5: Software Environment

This includes the supporting technologies of the project.

Chapter 6: Programming Techniques

This includes the techniques used to build the project.

Chapter 7: Implementation

This includes the coding done for the implementation of the project.

Chapter 8: Testing

The implementation details like algorithm and some code are being explained to this chapter.

Chapter 9: Results and Snapshots

CONCLUSION

REFERENCES

CONTACT DETAILS

1.2 Existing System.

The existing system for counterfeit product detection often relies on conventional methods that are susceptible to manipulation and error. Typically, manufacturers employ holograms, serial numbers, or physical labels on their products to establish authenticity. Consumers are then required to manually verify these features, which can be cumbersome and subject to human error. Moreover, counterfeiters have become increasingly skilled at replicating these physical attributes, making it challenging for consumers to differentiate genuine from counterfeit goods.

In addition to these limitations, the existing system lacks transparency and real-time tracking. Once a product leaves the manufacturer's facility, its history becomes difficult to trace. This opaqueness not only hampers consumers' ability to verify product authenticity but also makes it challenging for manufacturers to identify and counteract counterfeit production. Furthermore, the current system places a significant burden on consumers, who must invest time and effort in verifying products, often without a fool proof method to do so. This uncertainty erodes trust and can lead to financial losses and safety risks.

1.2.1 Disadvantages

- **Lack of Transparency:** Existing systems often lack transparency, making it difficult to track products throughout the supply chain.
- **Data Tampering:** Centralized databases are vulnerable to data tampering, which can lead to fake products being introduced into the market
- **Inefficient Tracking**: Traditional tracking methods can be time-consuming and prone to errors.
- **Limited Visibility:** Existing systems often provide limited visibility into the supply chain, making it challenging to identify counterfeit products.

CHAPTER 2

LITERATURE SURVEY

1.Title: "A comparative study: Blockchain technology utilization benefits, challenges, and functionalities"

Author: Ali, Omar, et al

Year: 2021

Publisher: IEEE Access 9

Description: It gives us a through explanation of blockchain technology. It provides comprehensive details regarding the blockchain's history, advantages, disadvantages, architecture, and security. Blockchain is a powerful toolbox — not a universal hammer. When multiple, partially mistrusting parties need shared, auditable records or when new token-driven business models are required, blockchain can deliver substantial value. However, it carries trade-offs (performance, privacy, governance complexity) that make traditional databases or hybrid approaches preferable in many cases. Effective adoption depends on carefully matching architecture to the use-case, managing off-chain/on-chain boundaries, and planning governance and compliance from day one.

2. Title: "Identifying Fake Product Through a QR Code Based Blockchain System"

Author: P.LokeshReddy, A.MuniKarthik, M.Pavitra, Mrs.M.S.Soundarya,M.E.,

Year: 2022

Publisher : IRJET

Description: - The production and marketing of counterfeit or counterfeit items and products exposes consumers to financial, health and safety risks. It also affects the business growth of companies and businesses through loss of revenue, bad products, losses and price changes, many names are forced to use the money for legal protection. Use camera scanners to identify counterfeit products and link the product's QR code or barcode to blockchain blocks. If the numbers on the work match the numbers in the library, a notification is sent to the customer informing them about the originality of the work. If the customer accepts the offer, a notification will also be sent to the manufacturer stating where the customer purchased the product, if the number on the product does not match the number in the information. This approach aims to reduce counterfeiting by ensuring that consumers do not rely on retailers to decide whether the product is genuine

or not. Product tracking using blockchain technology involves creating tamper-proof and immutable information about products from production to delivery.

3. Title: "QR based Fake Product Detection using Blockchain"

Author: Rivaa Vadher, Rishith Vadher

Year: 2024

Publisher: IJIRT

Description: The proliferation of counterfeit products poses a significant challenge to consumers and brand owners alike. Current software solutions for counterfeit detection lack the robustness and transparency needed to combat this pervasive issue. This project proposes a novel approach to address this problem by leveraging blockchain technology in conjunction with QR codes. Our software solution combines blockchain technology and QR codes to create a secure and transparent anti counterfeiting system. Each authentic product is assigned a unique QR code linked to a blockchain-based ledger. When consumers scan the QR code, they can instantly verify the product's authenticity and access detailed information about its origin, manufacturing, and distribution history. The immutable nature of blockchain ensures data integrity, making it nearly impossible for counterfeiters to manipulate or replicate. By integrating blockchain with QR codes, this software enhances consumer confidence and empowers them to make informed purchasing decisions. Furthermore, brand owners gain real-time insights into their supply chains, enabling them to identify and mitigate counterfeit production more effectively.

4.Title: **"Detection of Counterfeit Products using Blockchain"**

Author: Kunal Wasnik and Isha Sondawle

Year:2022

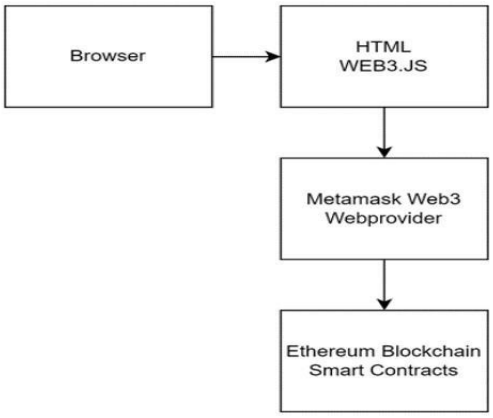
Publisher :ICACC

Description: There are many fake products in the existing supply chain. It is necessary to have a system for end user to check all details about product that they are buying so that the customer can check if the product is genuine or not. In recent years, Counterfeit products play an important role in product manufacturing industries. This affects the company name, sales, and profit of the companies. Block technology is used to identification of real products and detects fake products. Blockchain technology is the distributed, decentralized and digital ledger that stores transactional information in the form of blocks in many database/node computers which is connected with the chains.

Blockchain technology is secure as the data stored once in the chain is immutable therefore any block cannot be changed or hacked. By using Blockchain technology, customers or users do not need to rely on third party users for confirmation of product authenticity and safety. Our System provides the emerging technology of web use cases, Quick Response (QR) codes provide a robust technique to fight the practice of counterfeiting the products. Counterfeited products can be detected using a QR code scanner, where a QR code of the product is linked to Blockchain.

2.1 Proposed System:

The proposed application system aims to combat the increasing prevalence of counterfeit goods by utilizing blockchain and QR codes. The system will store the product's source chain and maintain ownership histories, allowing buyers to access comprehensive product details and make informed purchasing decisions. QR codes will be used to validate products and add product information, while blockchain technology ensures the integrity of the stored data, preventing unauthorized alterations. By leveraging these technologies, the system will effectively identify counterfeit products, enhancing trust and transparency in the market place.



2.1.1Advantages

- **Immutable Product History:** Blockchain technology creates an immutable record of a product's history, ensuring its authenticity.
- **Real-time Tracking**: The proposed system enables real-time tracking of products throughout the supply chain.

- **Tamper-Proof:** Blockchain's decentralized and immutable nature makes it difficult for counterfeiters to tamper with product information.
- **Increased Transparency:** The proposed system provides increased transparency, enabling stakeholders to verify product authenticity.

2.2 Objectives of the Project

1. Ensure Product Authenticity:

To verify whether a product is original or fake by scanning a unique QR code linked to tamper-proof blockchain records.

2. Establish a Transparent Supply Chain

To record every stage of the product lifecycle (manufacturing, packaging, shipping, retailing) on a blockchain so all stakeholders have a transparent view.

3. Provide Tamper-Proof Data Storage

To store product information (batch no., manufacturing date, serial details, ownership) on blockchain, where data cannot be modified or deleted.

4. Enable Real-Time Verification

To allow customers, retailers, and authorities to instantly verify product authenticity using a mobile-based QR scanner.

5. Improve Trust Between Manufacturer & Consumers

To build customer confidence by providing a reliable method to check product genuineness before purchase.

CHAPTER 3

SYSTEM REQUIRMENT SPECIFICATION

3.1 Functional Requirements

3.1.1 Product Registration:

Manufacturers can register products on the blockchain with a unique product ID (e.g., QR code, RFID).

3.1.2 Blockchain Entry Creation:

Each product entry includes key metadata (e.g., product name, serial number, manufacturer ID, timestamp, batch number).

3.1.3 Authenticity Verification:

Customers or retailers can scan a product's code to verify its authenticity via the blockchain ledger.

3.1.4 Immutable Ledger:

Once recorded, product data is immutable (can’t be altered), ensuring trust and traceability.

3.1.5 User Roles and Access Control:

Different roles (e.g., manufacturer, retailer, consumer, auditor) with appropriate permissions.

3.1.6 Transaction Logging:

Record all interactions with product data (e.g., verification attempts, ownership transfers).

3.1.7 Ownership Transfer:

Allow authenticated users (like suppliers) to transfer ownership of a product on the blockchain (supply chain tracking).

3.1.8 Counterfeit Reporting:

Users can report potentially fake products, which are logged and flagged for review.

3.2 Non-Functional Requirements

3.2.1 Hardware Requirements

- **Processor**: Intel Core i5
- **RAM:** 4GB

3.2.2 Software Requirements:

1. Operating System

- Windows 11

2.Blockchain Development Tools

- **Ganache**: A personal blockchain for Ethereum development.
- **Solidity**: Programming language for writing smart contracts.
- **MetaMask**: A browser extension for interacting with the Ethereum blockchain.

3. Integrated Development Environment (IDE)

- **Visual Studio Code (VS Code)**: A source-code editor with support for blockchain development.

4. Backend Framework

- **Flask**: A Python web framework for building the backend.

5. Frontend Technologies

- **HTML/CSS/JavaScript**: For building the user interface.
- **ReactJS**: A JavaScript library for building user interfaces.

6. Database

- **SQLite**: A lightweight database for storing product data.

7. Additional Tools

- **Anaconda**: A distribution of Python and R for scientific computing.
- **MySQL**: A relational database management system.

CHAPTER 4

SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

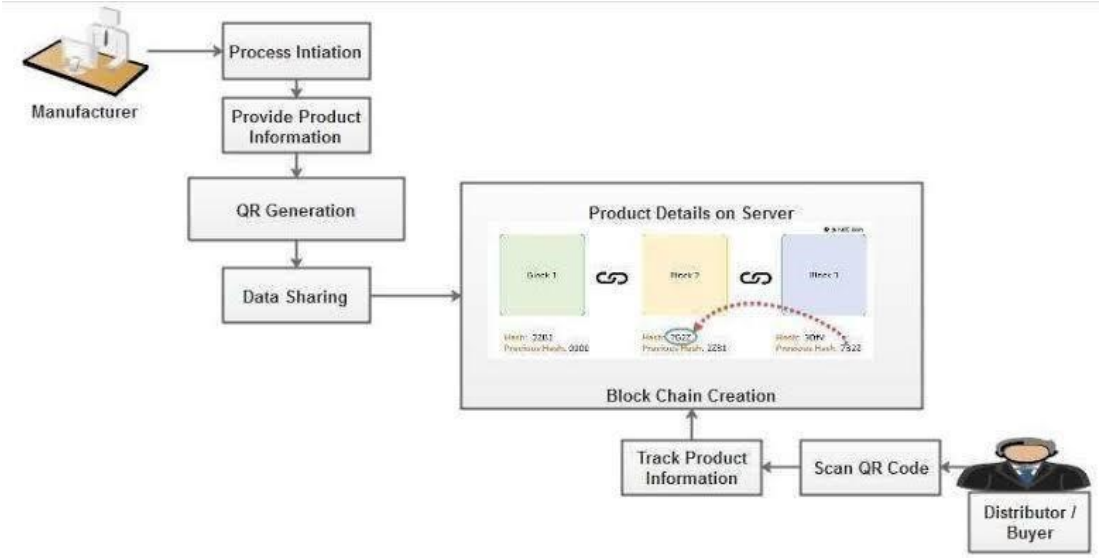


Fig 4.1.1 System Architecture

4.2 BLOCK DIAGRAM

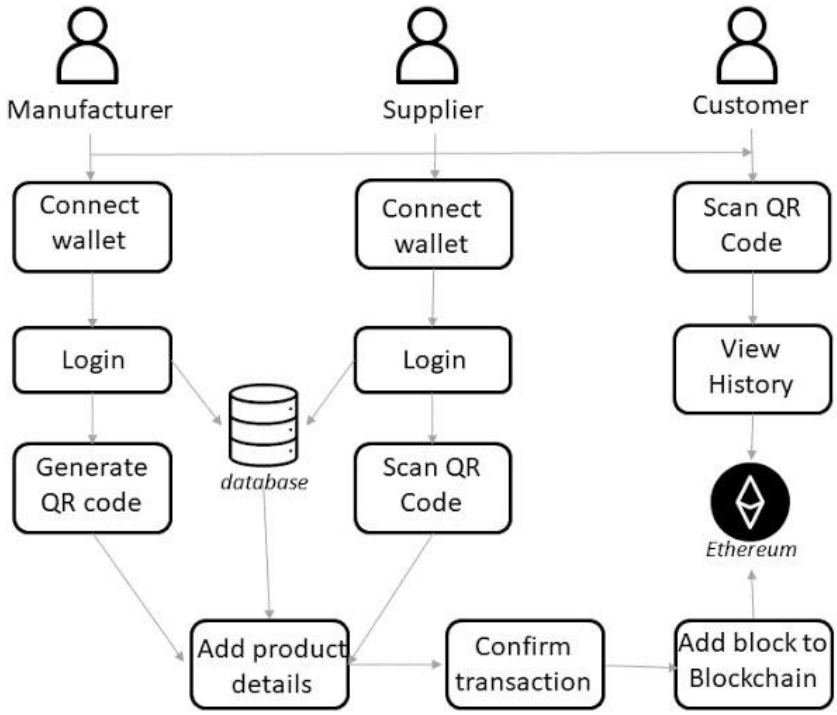


Fig 4.2.1 Block Diagram

4.3 SEQUENCE DIAGRAM

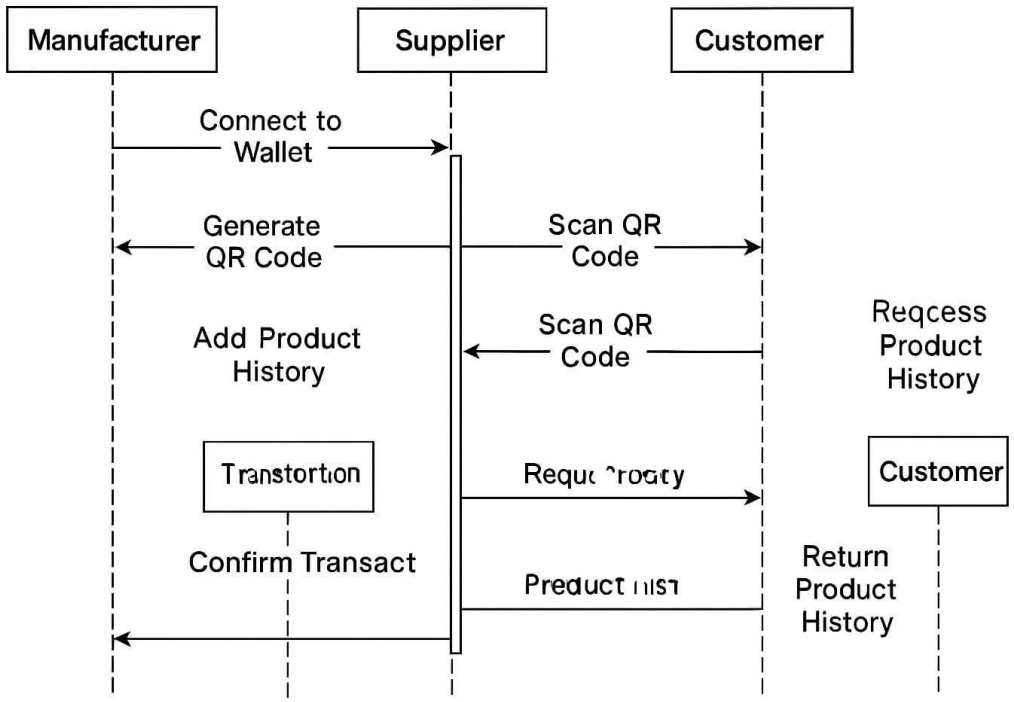


Fig 4.3.1 Sequence Diagram

4.4 USE CASE DIAGRAM

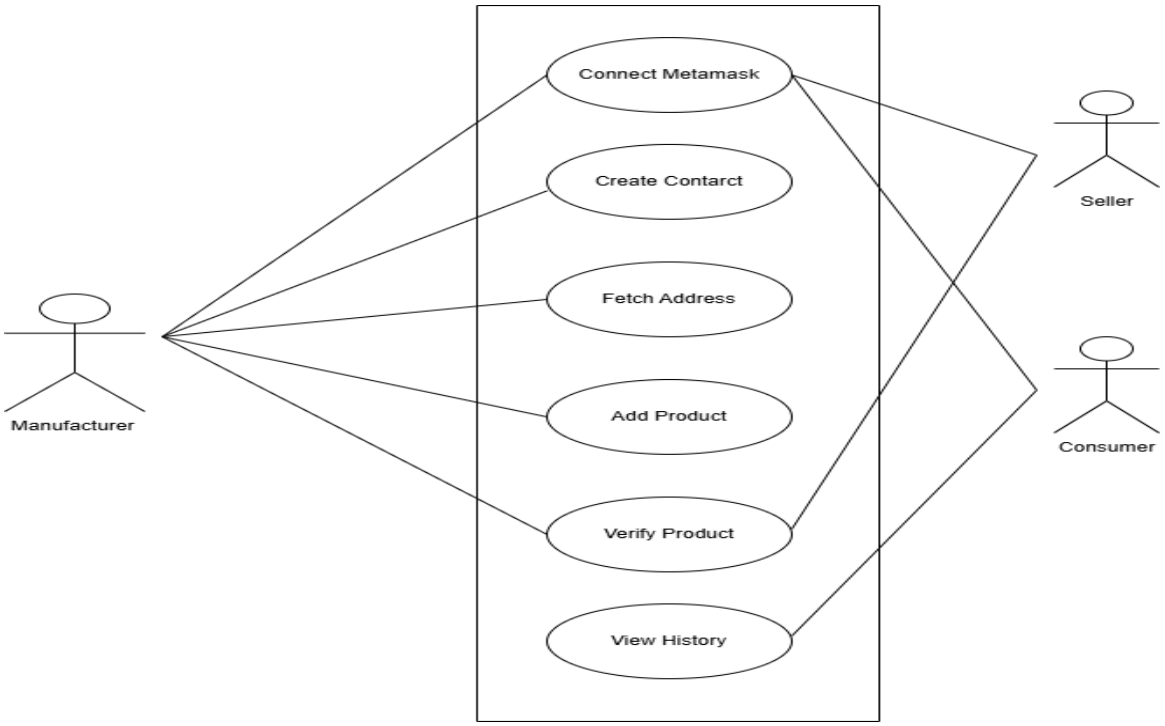


Fig 4.4.1 USECASE DIAGRAM

Entities Of The System

1. MANUFACTURER

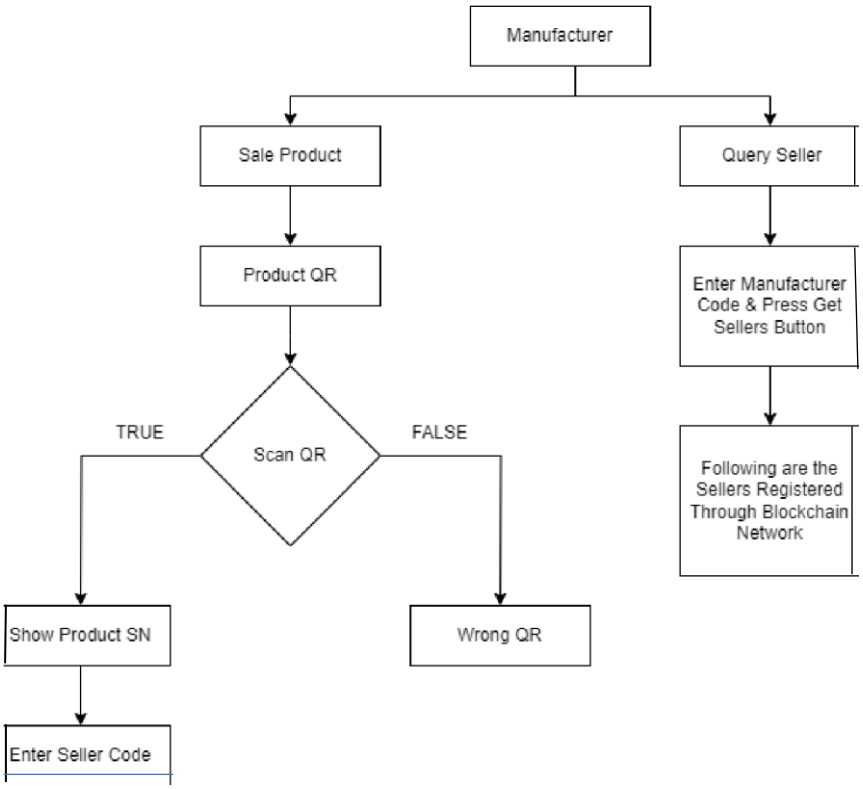
The manufacturer plays a crucial role in the proposed system model by adding authorized sellers, adding products with detailed information, updating ownership histories when products are sold, and querying the system to retrieve information about registered sellers. These functions contribute to maintaining the integrity of the supply chain, combating counterfeit goods, and providing transparency to buyers in making informed purchasing decisions. The manufacturer's functions in the system include:

1.Adding Seller: The manufacturer has the role of adding authorized sellers to the system. This involves registering sellers who are authorized to distribute and sell the manufacturer's products. By adding sellers to the system, the manufacturer establishes a trusted network of authorized distribution channels.

2.Adding Product: The manufacturer is responsible for adding their products to the system. This involves providing comprehensive details about each product, such as its name, description, unique identifiers, manufacturing information, and any other relevant information. By adding products to the system, the manufacturer enables buyers to access accurate and detailed information about the products they are considering purchasing.

3.Sale Product: When a manufacturer sells a product to an authorized seller, they update the ownership history of the product in the system. This ensures that the product's ownership chain is recorded and can be traced back to the manufacturer. By maintaining ownership histories, the system enables buyers to verify the authenticity and legitimacy of a product by checking its ownership trail.

4.Query Seller: The manufacturer can make queries to find sellers registered under their brand. This function allows the manufacturer to retrieve information about the authorized sellers associated with their products. It helps the manufacturer monitor the distribution network, track sales, and ensure that the products are being sold through authorized channels.

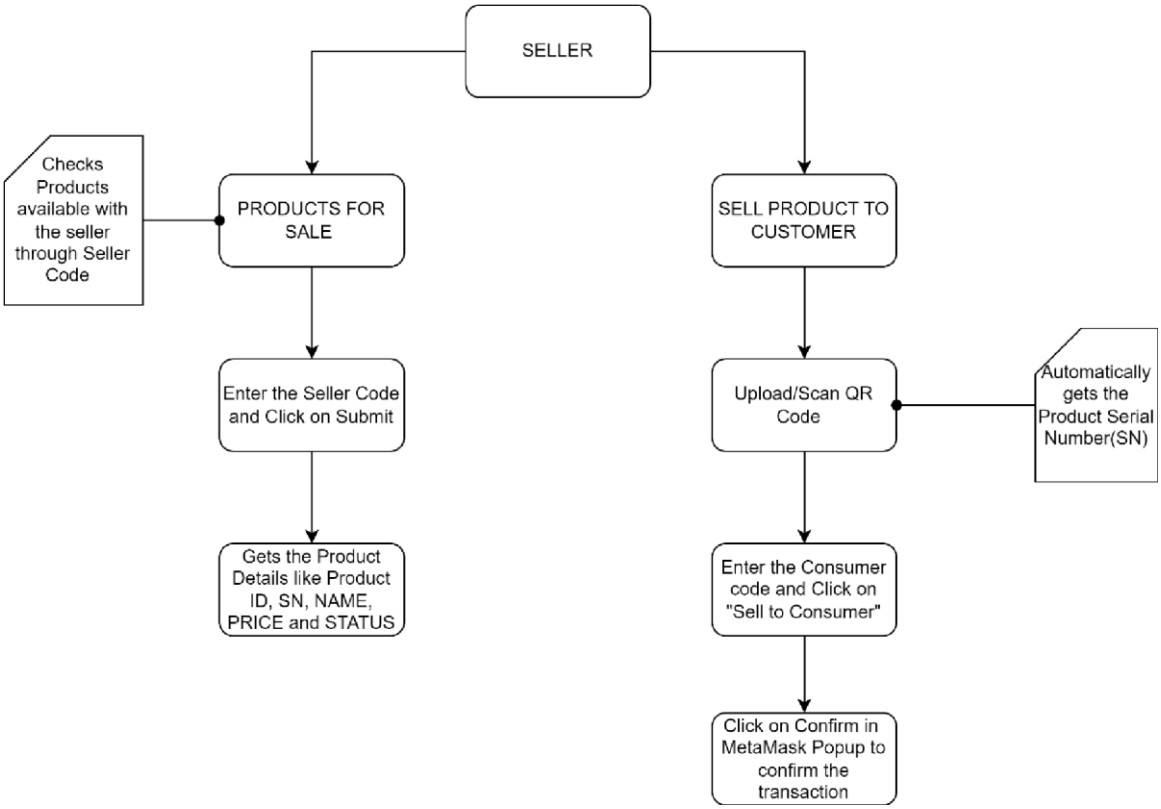


2. SELLER

The seller's roles in the proposed system model involve checking the availability of products for sale and facilitating the sale process to consumers. They rely on the information provided by the manufacturer and utilize the system's features, such as QR code validation and blockchain-backed data integrity, to ensure the authenticity and traceability of the products they sell. The seller's functions are as follows:

1.Checking Available Products: The seller can access the system to check for available products for sale. This involves querying the system to retrieve a list of products associated with their authorized manufacturer. By checking the available products, the seller can determine which products they can offer to consumers.

2.Selling the Product to Consumers: Once the seller identifies a product for sale, they can facilitate the transaction with the consumer. This includes providing product information, assisting with the purchase process, and handling the logistics of delivering the product to the consumer. The seller ensures that the transaction is completed securely and efficiently.



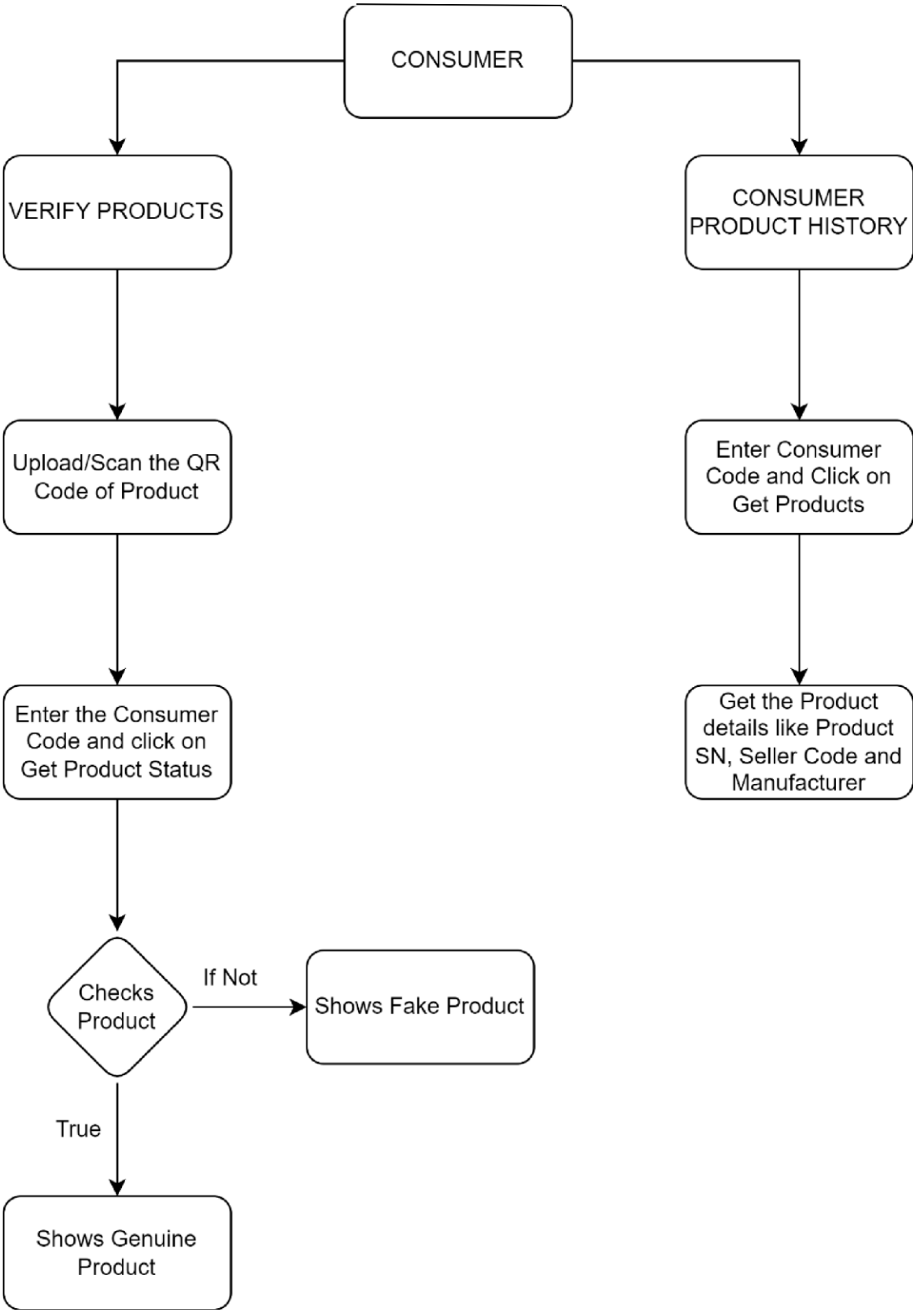
3. CONSUMER

The consumer's roles in the proposed system model involve verifying the authenticity of the product using the system's QR code verification feature and accessing the product's history to gain transparency and make informed decisions. By utilizing these functions, the consumer can protect themselves from counterfeit goods and ensure the integrity of their purchases.

The consumer's functions are as follows:

1.Verifying the Product: The consumer can use the system to verify the authenticity of the product they are interested in purchasing. This involves scanning or inputting the product's QR code, which triggers a verification process. This allows the consumer to ensure that they are buying genuine products and not falling victim to counterfeits.

2.Checking Product History: The consumer can access the product's history stored in the system. This includes information such as the product's origin, manufacturing details, previous owners (including the manufacturer and authorized sellers), and any relevant data associated with the product's journey through the supply chain. By checking the product's history, the consumer can gain insights into the product's authenticity, quality, and compliance with standards.


CONSUMER
VERIFY PRODUCTS
CONSUMER PRODUCT HISTORY
Upload/Scan the QR Code of Product
Enter Consumer Code and Click on Get Products
Enter the Consumer Code and click on Get Product Status
Get the Product details like Product SN, Seller Code and Manufacturer
Checks Product
If Not
Shows Fake Product
True
Shows Genuine Product

CHAPTER 5

SOFTWARE ENVIRONMENT

1. Ganache:

Ganache is a personal blockchain software that allows users to create their own Ethereum blockchain network for development and testing purposes.

- It provides a local blockchain environment that runs on the user's machine, allowing for faster and more efficient development without the need for interaction with the public Ethereum network.
- With Ganache, users can create multiple accounts with pre-funded Ether for testing purposes, making it easy to simulate real-world scenarios.
- It also offers a graphical user interface (GUI) that provides a comprehensive overview of the blockchain network, including account balances, transaction history, and block information.
- Ganache allows users to deploy and interact with smart contracts on the local blockchain network, enabling developers to test and debug their contracts before deploying them to the main Ethereum network.
- It provides additional features like automated contract testing, allowing developers to write tests to ensure the correctness and functionality of their smart contracts.
- Ganache also supports integration with other development tools and frameworks, such as Truffle, allowing for a seamless and efficient development workflow.

2. Truffle Suite

Truffle Suite is a development framework for building decentralized applications (DApps) and smart contracts on various blockchain platforms, including Ethereum. It offers a suite of tools that simplify the development, testing, deployment, and management of blockchain-based applications. Here are some of the basic functions and components of Truffle Suite:

- **Truffle Framework**: The core component of Truffle Suite is the Truffle Framework, which provides a development environment for creating and managing smart

contracts. It offers a project structure, build pipeline, and testing framework to streamline the development process.

- **Smart Contract Compilation:** Truffle Suite includes a Solidity compiler that automatically compiles your smart contracts into bytecode, which can be deployed on the blockchain. It supports the latest Solidity versions and allows you to configure compilation settings.

- **Contract Migration:** Truffle provides a migration system that helps in deploying smart contracts to the blockchain. Migration scripts are written in JavaScript and allow you to manage the deployment process, including deploying multiple contracts, handling dependencies, and updating contract versions.

- **Ganache Integration:** Truffle Suite seamlessly integrates with Ganache, a personal blockchain for local development. With Ganache, you can create a local blockchain network for testing and debugging your smart contracts, providing a simulated blockchain environment.

- **Contract Interaction:** Truffle provides a JavaScript library called Truffle Contract that simplifies the interaction with smart contracts from your DApp's frontend or backend. It abstracts away the complexities of interacting with the blockchain, providing a higherlevel API to send transactions, call contract functions, and retrieve contract state.

- **Network Management:** Truffle allows you to configure and manage connections to different blockchain networks, including local development networks like Ganache, as well as public testnets and mainnets. It simplifies the process of deploying contracts to different networks and switching between them.

- **DApp Development Tools:** Truffle Suite includes additional tools for DApp development, such as Truffle Boxes, which are pre-built project templates that provide boilerplate code for common use cases, and Drizzle, a front-end development framework that integrates with Truffle and simplifies state management and contract data fetching.

3.Metamask

Metamask is a web browser extension that serves as a bridge between the browser and the Ethereum blockchain. It provides a user-friendly interface for interacting with blockchain networks and managing Ethereum accounts. Here are its functionalities in reference to Ganache and Truffle: Product Authentication System Using Blockchain

- **Ethereum Wallet:** Metamask allows users to create and manage Ethereum wallets directly within their web browser. Users can generate and import Ethereum accounts, securely store private keys, and manage their digital assets.

- **Integration with Ganache:** Metamask can be configured to connect to the local blockchain network created with Ganache. This enables developers to interact with their locally deployed smart contracts directly from the browser, without the need for additional setups or deployments.

- **Smart Contract Interaction:** Metamask acts as a provider for web applications, enabling seamless integration with the Ethereum blockchain. It injects the web3.js library into web pages, allowing developers to interact with smart contracts deployed on the connected blockchain network, including those created with Truffle.

- **Transaction Management:** Metamask provides a user interface for users to initiate and manage transactions. When interacting with a decentralized application (DApp), Metamask prompts users to approve and sign transactions, ensuring secure and authenticated interactions with smart contracts.

4. NodeJs

Node.js is an open-source, server-side runtime environment that allows you to run JavaScript code outside of a web browser. It uses the V8 JavaScript engine, which is the same engine used by Google Chrome, to execute JavaScript code on the server. Node.js enables you to build scalable and high-performance web applications, command-line tools, and server-side APIs.

5. Solidity

Solidity is a programming language specifically designed for writing smart contracts on blockchain platforms, with Ethereum being the most prominent one. Smart contracts are self-executing contracts with the terms of the agreement directly written into code. Solidity allows developers to create these contracts, define their rules, and deploy them on the Ethereum Virtual Machine (EVM) or other compatible blockchain platforms.

CHAPTER 6

PROGRAMMING TECHNIQUES

1. Modular Programming

The system is divided into multiple modules such as:

- Manufacturer module
- Seller module
- Consumer module
- Blockchain verification module

Each module performs specific functions, making the code easier to maintain, debug, and extend.

2. Object-Oriented Programming (OOP)

OOP concepts are applied in smart contract development and JavaScript/Node.js implementation:

- **Encapsulation** – Data and functions related to blockchain transactions are grouped together.
- **Inheritance** – Common functionalities are reused across different components.
- **Abstraction** – Complex blockchain operations are hidden behind simple interfaces.

3. Event-Driven Programming

The application uses JavaScript and Web3.js to respond to user actions like scanning QR codes, verifying products, or connecting wallets.

Smart contracts also trigger events on the blockchain to notify users of completed transactions.

4. Client–Server Architecture

The system follows a client-server model, where:

The frontend (HTML, CSS, JavaScript) interacts with users.

The backend (Node.js, Web3.js, Truffle) communicates with the blockchain network.

5. Blockchain Programming

Smart contracts are developed using Solidity and deployed on Ethereum test networks.

This ensures:

- Decentralized data storage
- Tamper-proof product verification
- Transparency across the supply chain

6. Asynchronous Programming

JavaScript's asynchronous features (Promises and async/await) are used to handle blockchain transactions without freezing the interface.

7. Form Handling and Validation

User input data (like product code, manufacturer ID, etc.) is validated using JavaScript to prevent incorrect entries before storing data on the blockchain.

8. QR Code Integration

QR code generation and scanning techniques are used for encoding and decoding product details, ensuring quick and secure product verification.

9. Responsive Web Design

HTML5, CSS3, and Bootstrap are used to make the user interface responsive and accessible across all devices.

These programming techniques ensure that the system is **secure, modular, efficient, and user-friendly**, leveraging **blockchain technology** for transparent product verification.

CHAPTER 7

IMPLEMENTATION

The system is divided into multiple modules such as:

- Manufacturer module
- Seller module
- Consumer module
- Blockchain verification module

Each module performs specific functions, making the code easier to maintain, debug, and extend.

7.1 Algorithm

Step 1: Signup to blockchain as manufacturer or seller or consumer.

Step 2: Login to blockchain

if user is manufacturer

then register product and generate QR code

scan the QR along with location

deduct ETH for each product registration

record the transaction

Step 3: if user is seller

scan the QR along with location

deduct ETH from wallet for each scanning

update the details in blockchain

Step 4: if user is consumer

Verify the received product

By scanning the QR code

if image is QR code and generated by blockchain

than product details are shown

else

product details are not available

Step 6: Logout of the transactions.

7.2 Pseudo code

Manufacturer Module:

```
<!doctype html>

<html lang="en">

  <head>

    <title>Fake Product Identification</title>

    <meta charset="utf-8">

    <meta name="viewport" content="width=device-width, initial-scale=1, shrink-tofit=no">

    <!-- Navbar -->

    <nav class="navbar navbar-expand-lg navbar-dark">

      <div class="container-fluid">

        <a class="navbar-brand" href="index.html">Fake Product Identification

          <span>through Blockchain</span>

        </a>

        <button class="navbar-toggler" type="button" data-toggle="collapse" datatarget="#ftco-nav">

          <span class="fa fa-bars"></span>

        </button>

        <div class="collapse navbar-collapse" id="ftco-nav">

          <ul class="navbar-nav m-auto">

            <li class="nav-item"><a href="index.html" class="nav-link">Home</a></li>

            <li class="nav-item"><a href="addProduct.html" class="nav-link">Add Product</a></li>
```

```
        <li class="nav-item"><a href="addSeller.html" class="nav-link">Add Seller</a></li>

        <li class="nav-item"><a href="sellProductManufacturer.html" class="navlink">Sell Product</a></li>

        <li class="nav-item"><a href="querySeller.html" class="nav-link">Query Seller</a></li>

       </ul>

      </div>

    </div>

  </nav>

 </body>

</html>
```

Seller Module:

```
<!doctype html>

<html lang="en">

<head>

  <title>Seller Dashboard</title>

<body>

  <!-- Navbar -->

  <nav class="navbar navbar-expand-lg navbar-dark">

    <div class="container-fluid">

                    <a class="navbar-brand" href="index.html">Fake Product Identification <span>through Blockchain</span></a>

    <button class="navbar-toggler" type="button" data-toggle="collapse" datatarget="#ftco-nav">

      <span class="fa fa-bars"></span>
```

```
    </button>
    <div class="collapse navbar-collapse" id="ftco-nav">
      <ul class="navbar-nav m-auto">
        <li class="nav-item"><a href="index.html" class="nav-link">Home</a></li>
        <li class="nav-item"><a href="sellProductSeller.html" class="nav-link">Sell Product To Consumer</a></li>
        <li class="nav-item"><a href="queryProducts.html" class="nav-link">Products For Sale</a></li>
      </ul>
    </div>
  </div>
 </nav>
```

Consumer Module:

```
<!doctype html>
<html lang="en">
<head>
 <meta charset="utf-8">
 <meta name="viewport" content="width=device-width, initial-scale=1, shrink-tofit=no">
 <title>Fake Product Identification</title>
<body>
<!-- Navbar -->
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
 <div class="container">
   <a class="navbar-brand font-weight-bold" href="index.html">Fake Product ID</a>
```

```
<button class="navbar-toggler" type="button" data-toggle="collapse" datatarget="#navbarNav">
  <span class="fa fa-bars"></span>
 </button>
 <div class="collapse navbar-collapse" id="navbarNav">
  <ul class="navbar-nav ml-auto">
   <li class="nav-item"><a class="nav-link" href="index.html">Home</a></li>
<li class="nav-item"><a class="nav-link" href="consumerPurchaseHistory.html">Purchase History</a></li>
   <li class="nav-item"><a class="nav-link" href="verifyProducts.html">Verify Product</a></li>
  </ul>
 </div>
</div>
</nav>
```

Blockchain verification Module:

```
// SPDX-License-Identifier: MIT

pragma solidity >=0.4.22 <0.9.0;

contract Migrations {   address public

owner = msg.sender;   uint public

last_completed_migration;  modifier

restricted() {     require(

    msg.sender == owner,

    "This function is restricted to the contract's owner"
```

```
    );

    _;

  }

 function setCompleted(uint completed) public restricted {

last_completed_migration = completed;
```

CHAPTER 8

TESTING

8.1 Scope Of Testing

Testing is the process of systematically evaluating a system, application, or component to identify any discrepancies between the expected and actual behaviour.The primary goal of testing is to ensure that the software or system meets specified requirements, functions correctly, and delivers the desired outcomes.

Key aspects of testing include:

1. **Verification:** Confirming that the product is being developed according to the Specified requirements. It involves activities such as reviews and inspections to catch defects early in the development process.
2. **Validation:** Ensuring that the end product satisfies the intended use and meets the needs of the users. This involves dynamic testing where the software is executed to assess its behaviour.
3. **Debugging:** The process of identifying, isolating, and fixing defects or issues in the software. Debugging is usually performed by developers during the development phase.

8.2 Types Of Testing

1 Unit Testing:

Unit Testing Definition: Verifies individual code units, usually functions or methods, to ensure correct behavior.

Also Known As**:** Component testing.

Testing Scope: Focuses on specific code sections, typically at the function or method level.

Object-Oriented Focus: Targets classes, checking methods and properties for correct interaction.

Minimal Tests**:** Should include constructors and destructors to ensure proper object lifecycle.

Benefits: Early bug detection, easier debugging, safer refactoring, and improved documentation through tests.

2 Integration Testing

Integration Testing Definition: Verifies interfaces between software components against the design.

Purpose: Detects defects in interfaces and interactions between integrated modules.

3 Validation Testing

Validation testing ensures software functions meet customer expectations

Also known as software quality control

4 User Acceptance Testing

User Acceptance Testing (UAT) is the final phase of the software testing process where the actual users validate the system to ensure it meets their requirements and works as expected in real-world scenarios.The performance of an acceptance test is essentially the user's show, meaning that end users take the lead in executing these tests to confirm the software's readiness for deployment.This phase is critical because it verifies that the system is not only technically sound but also user-friendly and aligned with business needs.

8.3 Test Cases

Test Case	1
Name of Test	Manufacturer
Input	Takes the data from the manufacturer
Expected output	Will generate code
Actual output	Gives the information to the following gives to distributer
Result	Successful

Table 8.3.1 Manufacturer Test case

Test Case	2
Name of Test	Seller
Input	Takes the data from the manufacturer
Expected output	It will access and provide its information respectively
Actual output	Takes the input and provide the information to the next part
Result	Successful

Table 8.3.2 Seller Test case

Test Case	3
Name of Test	Consumer
Input	Get the output from the retailer and family gives
Expected output	QR code will be generated and all information will be given
Actual output	QR code is generated and the consumer will get history
Result	Successful

Table 8.3.3 Consumer Test case

CHAPTER 9

RESULTS AND SNAPSHOTS

ADD SELLER

1) Navigate to the "Add Seller" Tab: Log in to the web page as a manufacturer and navigate to the "Add Seller" section or tab.

2) Fill in Seller Details: In the "Add Seller" form, enter the required details of the seller, such as their business name, contact information, address, and any other necessary information. Ensure that the form captures all the relevant data needed to add the seller to the system.

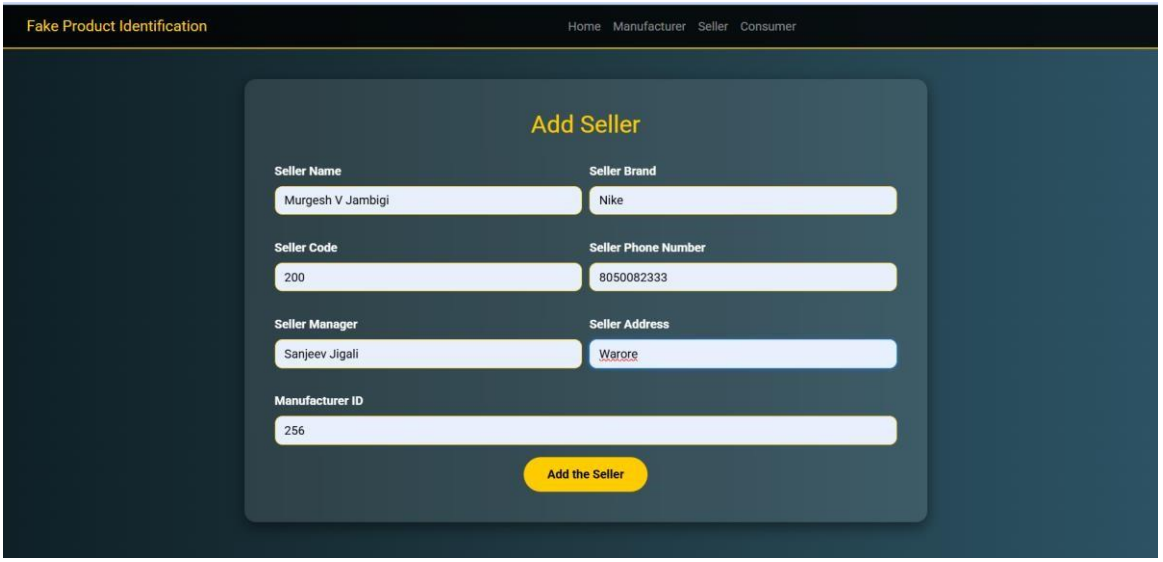


Fig 9.1: Add Seller Tab

3) Connect Metamask Account: Click on the "Connect Metamask" button or link provided on the web page. This will prompt you to log in to your Metamask account and grant permission for the web page to access your Ethereum address.

4) Confirm Transaction: Once connected to your Metamask account, review the details of the transaction on the Metamask pop-up window. Confirm the transaction by clicking the "Confirm" button. This will initiate the transaction to add the seller to the system.

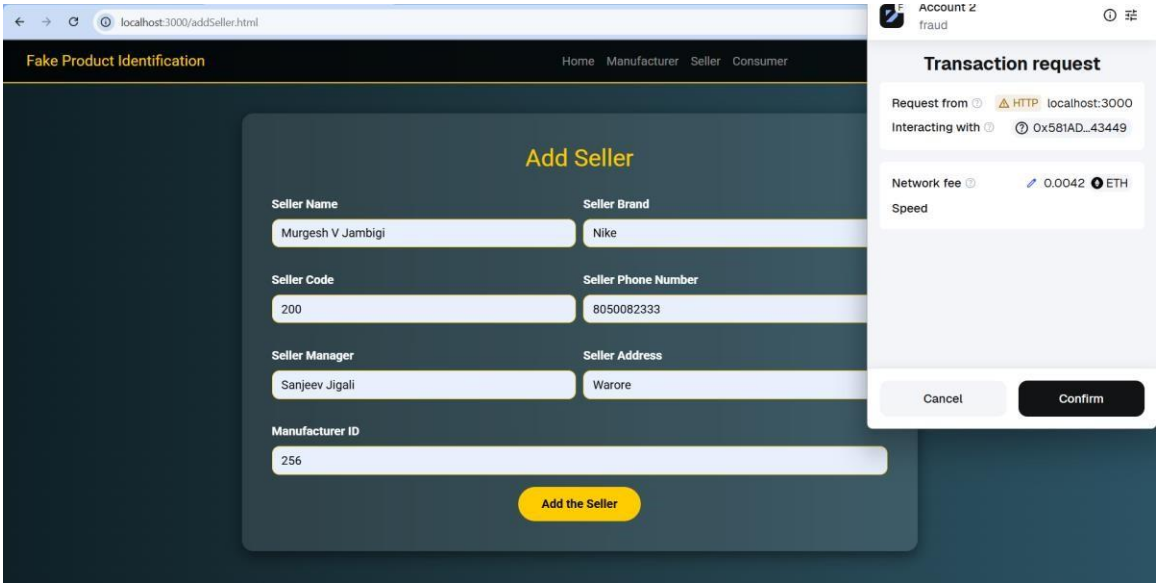


Fig 9.2: Seller Confirmation By Metamask

5) Wait for Transaction Confirmation and Success Notification: After confirming the transaction, wait for the transaction to be processed and confirmed on the blockchain. This may take a few seconds to a few minutes depending on the blockchain network's speed. Once the transaction is successfully confirmed, you will receive a notification or confirmation message on the web page indicating that the seller has been added to the system. This confirms that the seller's details are now recorded on the blockchain and they are officially part of the system.

ADD PRODUCT

1) Navigate to the "Add Product" Tab: Log in to the web page as a manufacturer and navigate to the "Add Product" section or tab.
2) Fill in Product Details: In the "Add Product" form, enter the required details of the product, such as its name, description, unique identifier, source chain information, and any other relevant information. Make sure to provide accurate and comprehensive details about the product.

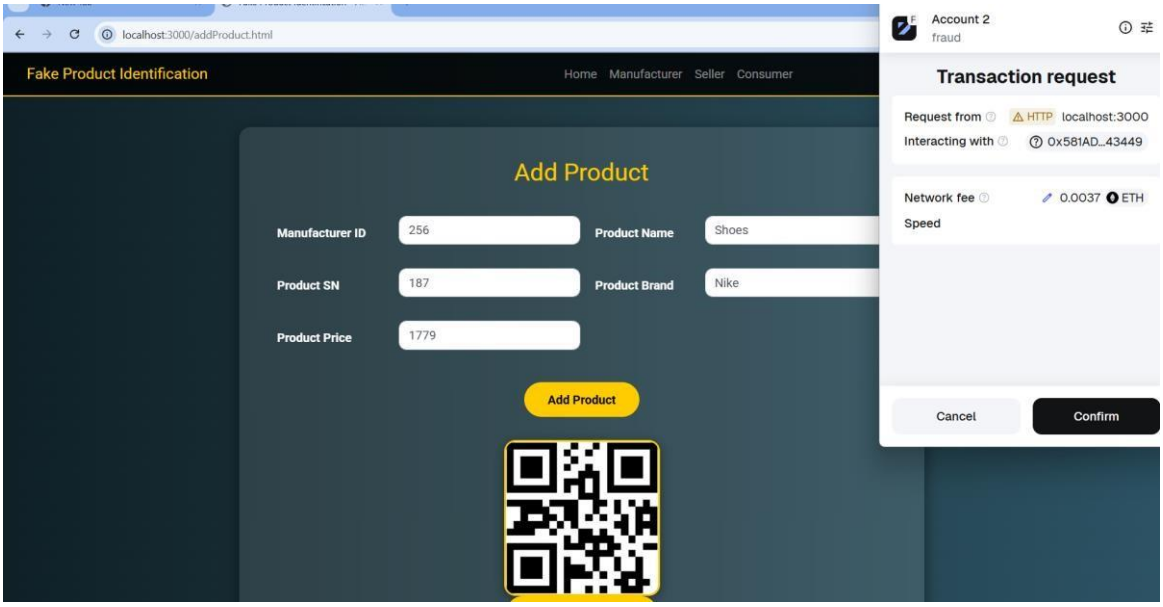


Fig 9.3: Add Product Tab

3) Confirm Transaction: Once connected to your Metamask account, review the details of the transaction on the Metamask pop-up window. Confirm the transaction by clicking the "Confirm" button. This will initiate the transaction to add the product to the system.

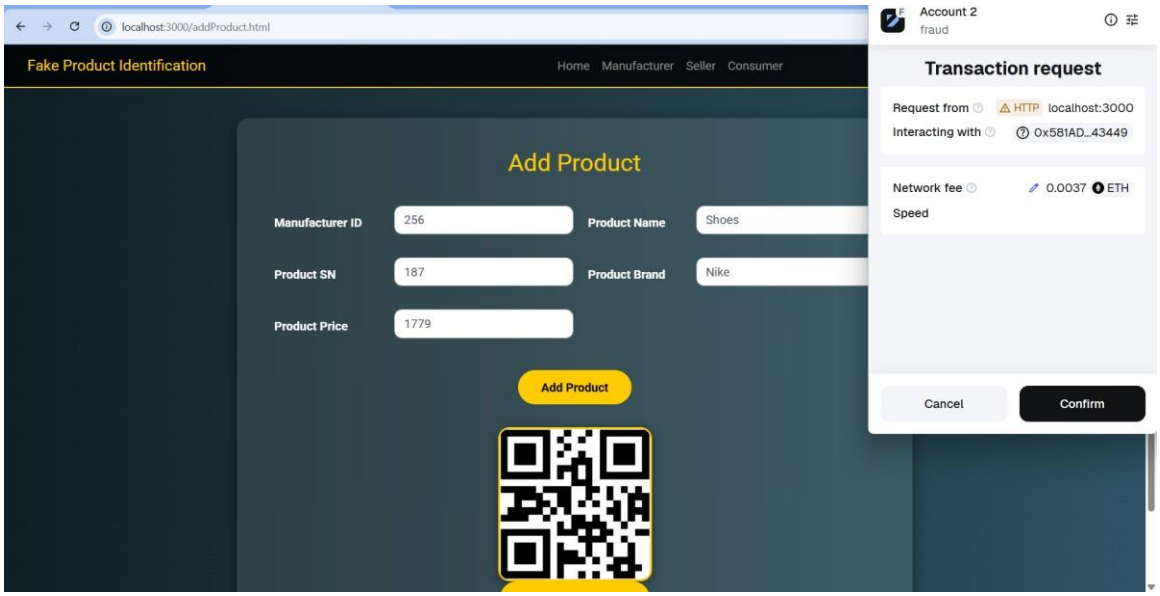


Fig 9.4: Product Confirmation

4) Wait for Transaction Confirmation and Success Notification: After confirming the transaction, wait for the transaction to be processed and confirmed on the blockchain. Once the transaction is successfully confirmed, you will receive a notification or confirmation message on the web page indicating that the product

has been added to the system. This confirms that the product's details are now recorded on the blockchain and it is officially part of the system.

SELL PRODUCT TO SELLER (MANUFACTURER)

1) Navigate to the "Sell Product" Tab: Log in to the web page as a manufacturer and navigate to the "Sell Product to Seller" section or tab in Manufacturer section.
2) Scan QR Code: Use the QR code scanner feature on the web page to scan the QR code of the product. This QR code contains the product's unique identifier.

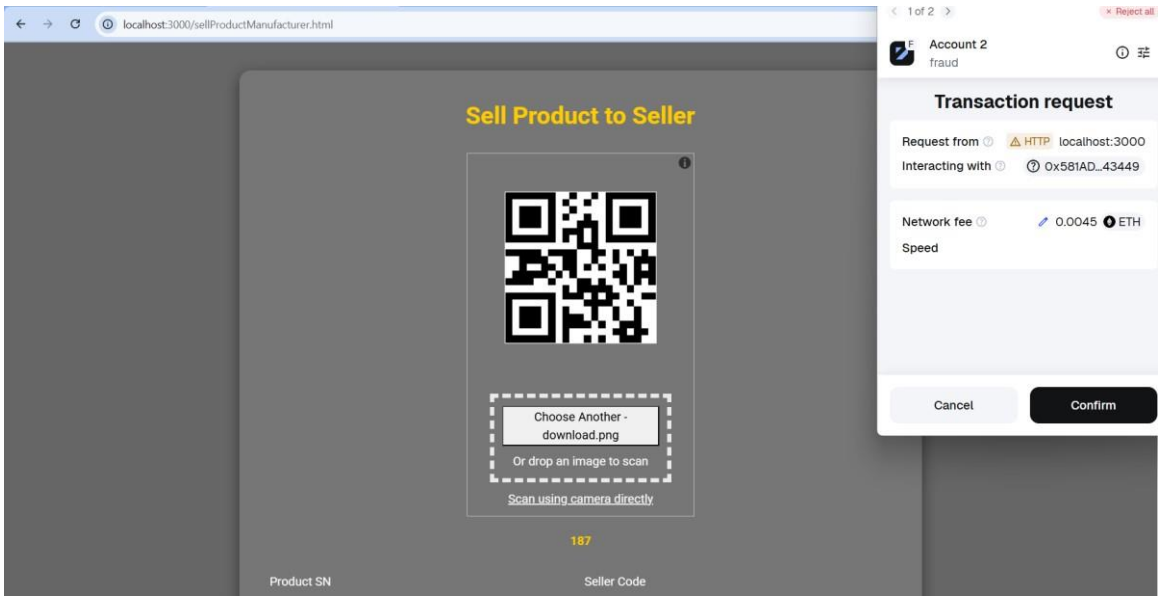


Fig 9.5: Sell Product to Seller

3) Enter Seller Code: In addition to scanning the product's QR code, enter the seller code associated with the seller you want to sell the product to. This code serves as an additional verification and ensures the correct seller is selected for the transaction.

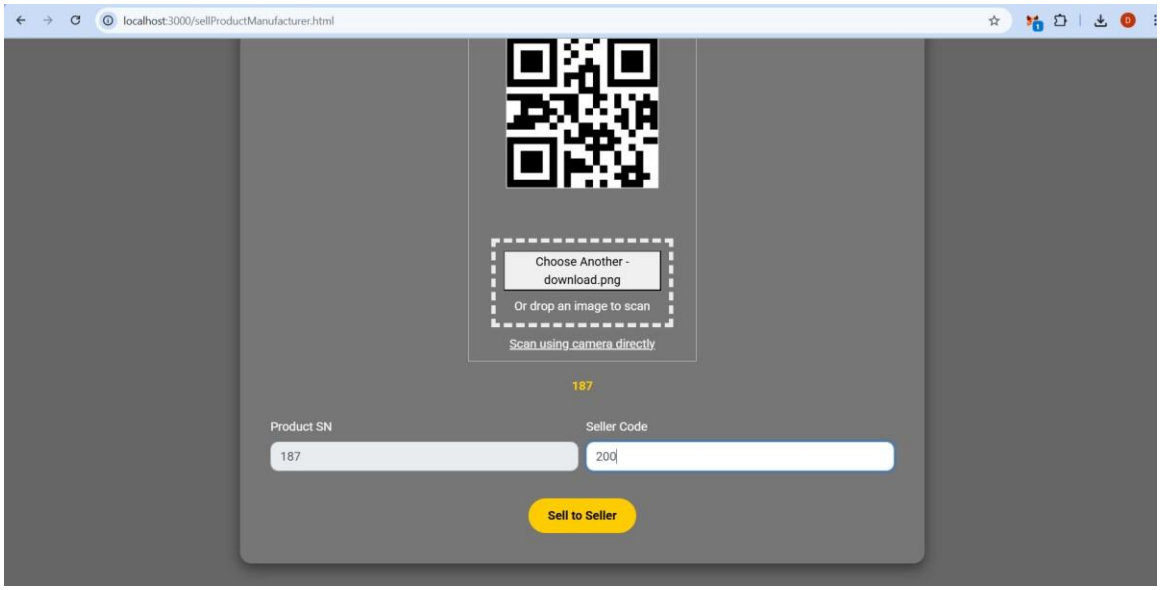


Fig 9.6: Confirm Product Sale to Seller

4) Confirm Transaction: After scanning the product's QR code and selecting the seller, connect your Metamask account and confirm the transaction. Wait for the transaction to be processed and confirmed on the blockchain, indicated by a notification or confirmation message on the web page. This ensures the secure transfer of ownership to the seller.

SELL PRODUCT TO CONSUMER (SELLER)

1) Scan Product QR Code: Use the web page's QR code scanner to scan the QR code of the product that the consumer wants to purchase.



Fig 9.7: Sell Product to Consumer

2) Connect Metamask Account: Click on the "Connect Metamask" button or link to log in to your Metamask account and grant permission for the web page to access your Ethereum address.
3) Confirm Transaction: Review the transaction details on the Metamask pop-up window and confirm the transaction. Wait for the transaction to be processed and confirmed on the blockchain, as indicated by a notification or confirmation message on the web page.

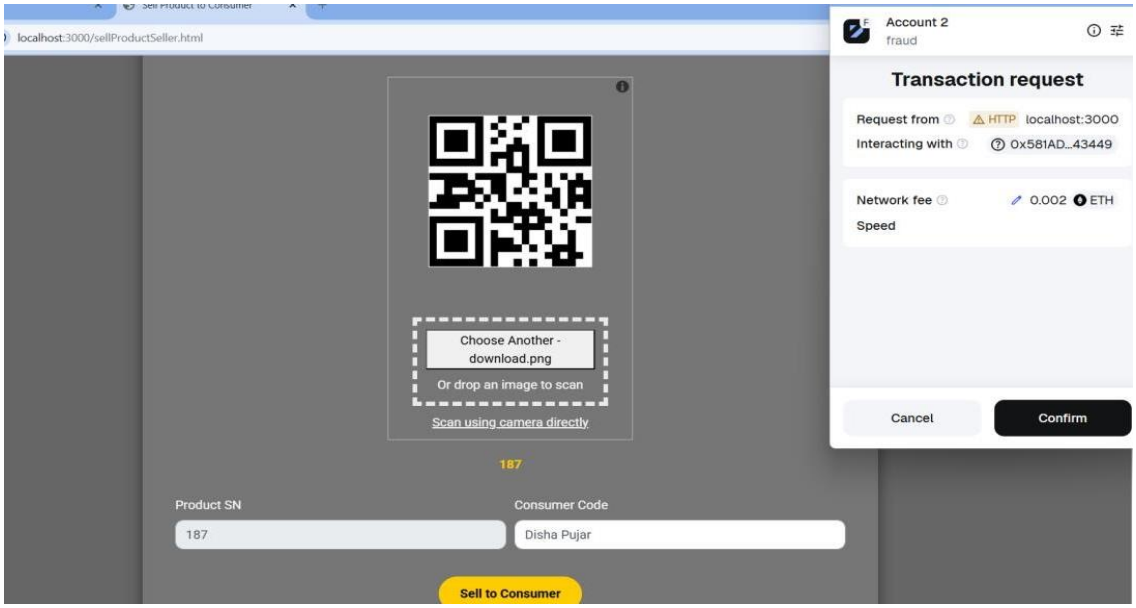


Fig 9.8:Confirm Product Sale to Consumer

4) Success Notification: Once the transaction is confirmed, the consumer will receive a notification or confirmation message on the web page indicating that the product has been successfully purchased. This confirms the transfer of ownership from the manufacturer to the consumer.

QUERY SELLERS REGISTER (MANUFACTURER)

This tab in Manufacturer section provides the list of all Seller under that manufacturer just by putting the “Manufacturer Id”.

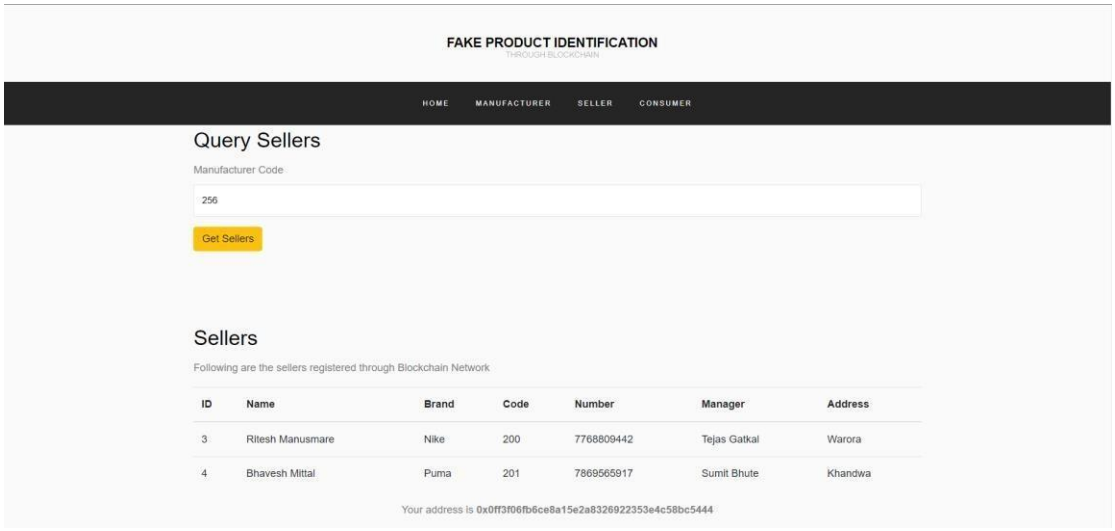


Figure 9.9: The Seller Register

QUERY PRODUCT REGISTER (SELLER)

This tab in Seller section provides the list of all Products under that Seller just by putting the “Seller Id”.

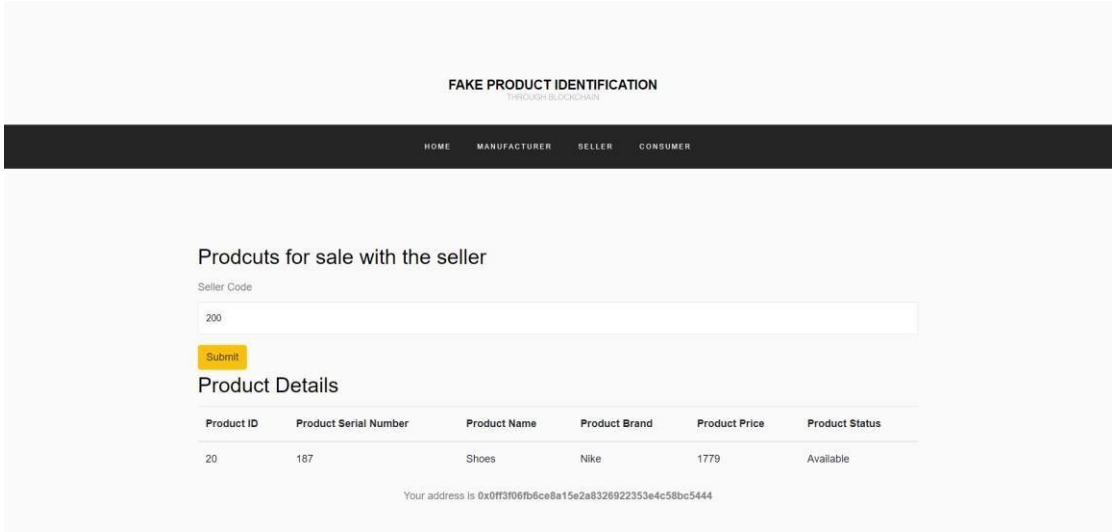


Figure 9.10: The Product Register

VERIFY PRODUCTS

1) Scan Product QR Code: Use a QR code scanner on your mobile device or a web page feature to scan the QR code on the product.
2) Connect Metamask Account: If the verification process involves interacting with the blockchain, click on the "Connect Metamask" button or link to log in

to your Metamask account and grant permission for the web page to access your Ethereum address.

3) Verify Product Authenticity: The Authenticity of the product will be either displayed as "Genuine" Or "Fake".

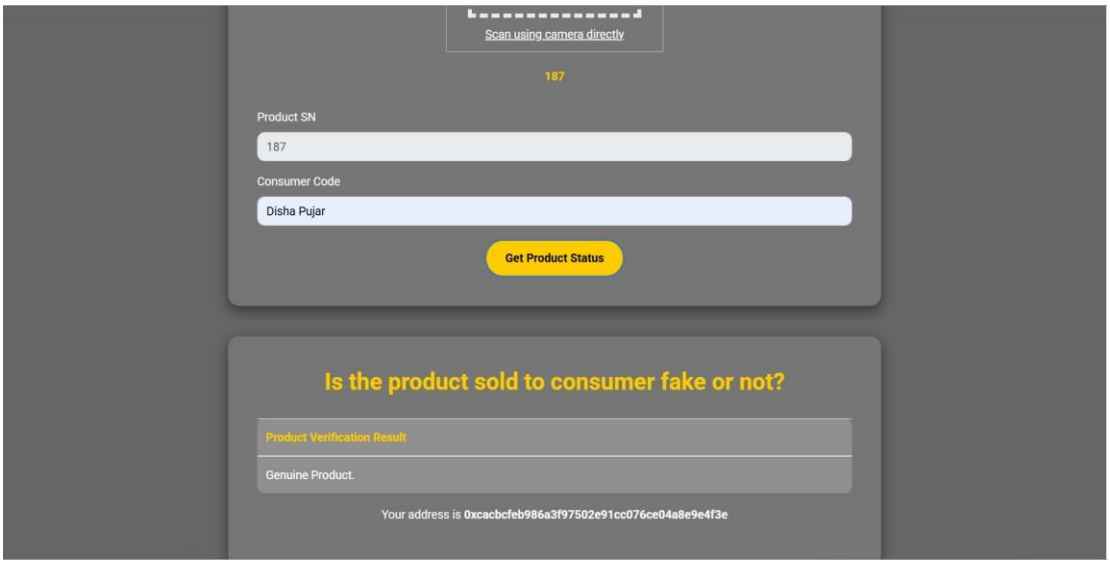


Fig 9.11: Product Status

CONSUMER PRODUCT HISTORY

This tab in Consumer section provides the list of all products purchased by the Consumer just by putting the “Consumer Id”.

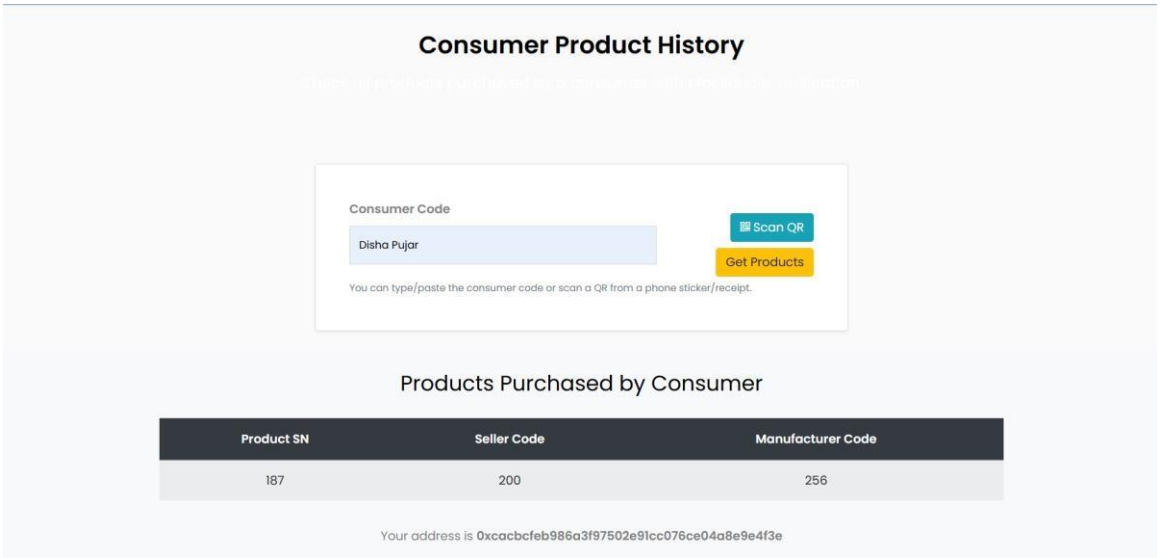


Fig 9.12:Consumer Product History